Image Captioning Model

BACHELOR OF ENGINEERING IN **COMPUTER ENGINEERING**

By

Sanchet Khemani / 21

Name of the Mentor

Prof. Sujata Khandaskar

Vivekanand Education Society's Institute of Technology,

An Autonomous Institute affiliated to University of Mumbai

HAMC, Collector's Colony, Chembur,

Mumbai-400074

University of Mumbai (AY 2025-26)

Abstract

This project implements a deep learning–based Image Caption Generator that automatically describes images in natural language. It combines Convolutional Neural Networks (CNNs) for extracting image features with Recurrent Neural Networks (RNNs) using LSTM units for generating text. Pre-trained CNN architectures such as VGG16, ResNet50, and DenseNet extract high-level image features, which are then fed into an LSTM decoder trained on paired image-caption data. The system demonstrates how Natural Language Processing (NLP) and Computer Vision (CV) can be integrated to bridge the gap between visual and textual understanding. The model is deployed in an interactive notebook environment to generate real-time captions, showcasing a practical multimodal AI application.

Background / Context

Image captioning is a challenging task at the intersection of Computer Vision and NLP. It requires recognizing visual elements in an image and describing them fluently in human language. This project demonstrates a CNN-LSTM fusion pipeline, combining:

- Vision Understanding: Using CNNs (VGG16, ResNet50) to extract image features.
- Language Generation: Using LSTMs to generate grammatically correct captions.

Applications include accessibility tools for visually impaired users, automated content tagging, and enhancing human-AI interaction.

Problem Statement

The goal is to design a system that generates meaningful captions for images. Challenges include:

- Understanding the visual semantics of an image (objects, actions, relationships).
- Capturing the syntactic structure of natural language to describe these semantics.
- Mapping high-dimensional visual data into structured textual output

Objectives

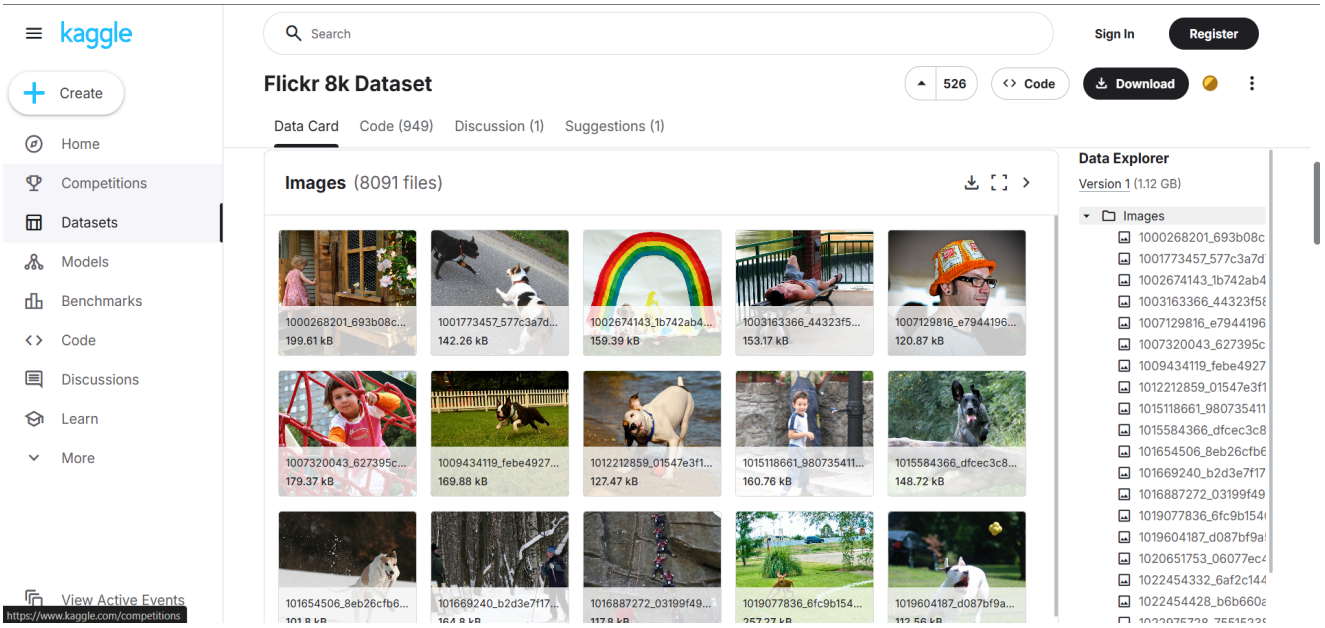
- Build a deep learning Encoder–Decoder model for image captioning.
- Extract meaningful visual features using pre-trained CNNs.
- Preprocess and clean text data for sequence learning.
- Train and evaluate the CNN-LSTM architecture.
- Deploy the model for interactive real-time caption generation.

Dataset Details

The project uses a standard image-caption dataset such as Flickr8k, Flickr30k, or MS COCO, consisting of: [Dataset_Link]

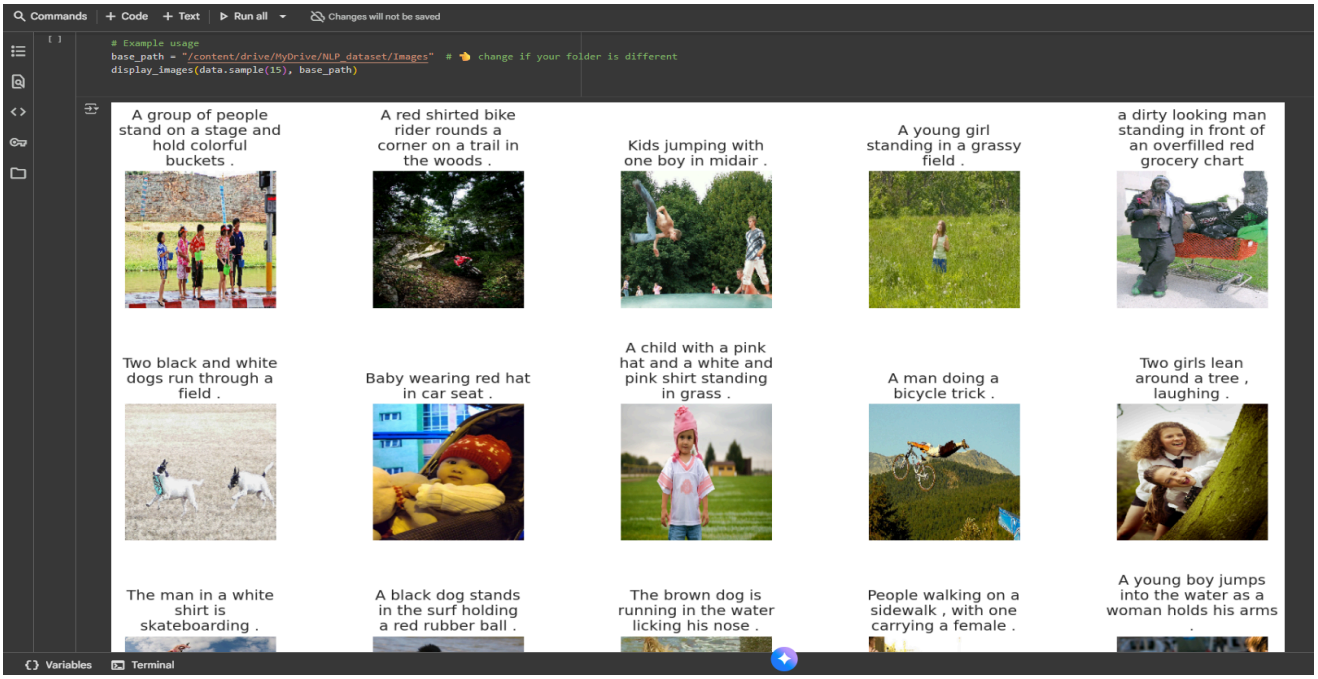
- 8,000 images (subset used for training).
- 5 captions per image.
- 85%-15% train-test split.

The output is a sequence of words forming a sentence for each image, rather than fixed class labels.



Methodology

1. Image Preprocessing:
   - Resize images to 224×224 pixels.
   - Normalize pixel values to [0,1].
   - Extract features using pre-trained CNNs without top layers.



2. Text Preprocessing:
   - Convert captions to lowercase.
   - Tokenize words and pad sequences.
   - Remove low-frequency words to reduce noise.
   - Wrap captions with <start> and <end> tokens.

```
# 6. Create train/test DataFrames
train = data[data['image'].isin(train_images)].reset_index(drop=True)
test = data[data['image'].isin(val_images)].reset_index(drop=True)

print("Training samples:", train.shape)
print("Validation samples:", test.shape)

# 7. Test tokenization
sample_seq = tokenizer.texts_to_sequences([captions[1]])[0]
print("Sample caption:", captions[1])
print("Tokenized:", sample_seq)
```

```
Vocabulary size: 8427
Max caption length: 35
Total images: 8091
Training samples: (34385, 2)
Validation samples: (6070, 2)
Sample caption: startseq girl going into wooden building endseq
Tokenized: [1, 18, 313, 63, 194, 116, 2]
```

3. Model Architecture:
   - Encoder (CNN): Extracts image features (4096 or 2048 dimensions).
   - Decoder (LSTM): Embedding layer → LSTM → Dense layer → predicts next word.
   - Fusion Layer: Concatenates image and text embeddings before prediction.

Model: "functional_1"

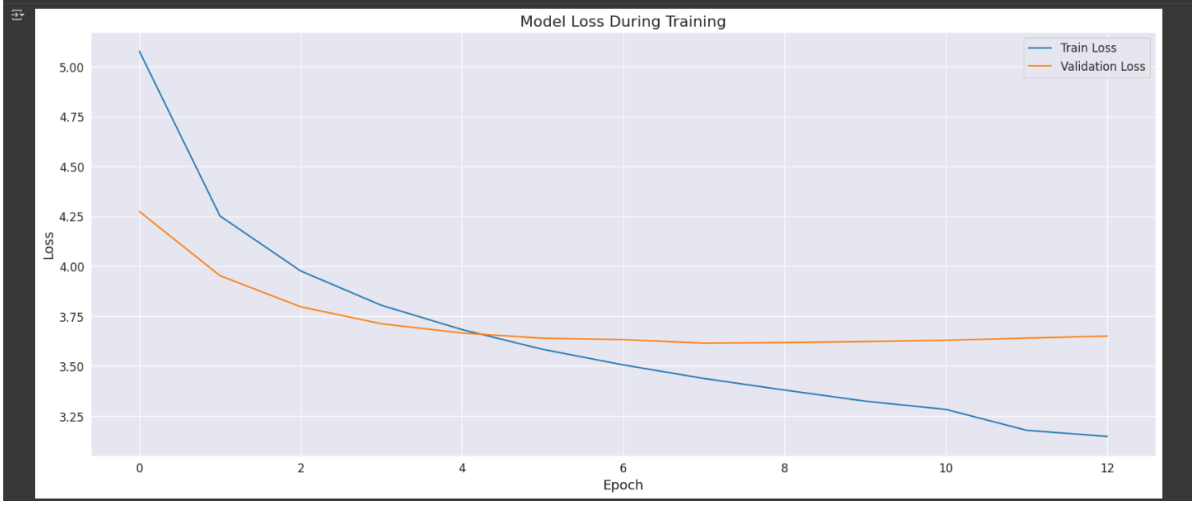
Layer (type)	Output Shape	Param #	Connected to
image_input (InputLayer)	(None, 1920)	0	-
dense (Dense)	(None, 256)	491,776	image_input[0][0]
sequence_input (InputLayer)	(None, 35)	0	-
reshape (Reshape)	(None, 1, 256)	0	dense[0][0]
embedding (Embedding)	(None, 35, 256)	2,157,312	sequence_input[0…
concatenate (Concatenate)	(None, 36, 256)	0	reshape[0][0], embedding[0][0]
lstm (LSTM)	(None, 256)	525,312	concatenate[0][0]
dropout (Dropout)	(None, 256)	0	lstm[0][0]
add (Add)	(None, 256)	0	dropout[0][0], dense[0][0]
dense_1 (Dense)	(None, 128)	32,896	add[0][0]
dropout_1 (Dropout)	(None, 128)	0	dense_1[0][0]
dense_2 (Dense)	(None, 8427)	1,087,083	dropout_1[0][0]

Total params: 4,294,379 (16.38 MB)
Trainable params: 4,294,379 (16.38 MB)
Non-trainable params: 0 (0.00 B)

Evaluation Metrics

- Technical Metrics: BLEU score, categorical crossentropy loss, and perplexity.
- User-Centric Metrics: Caption readability, generation speed

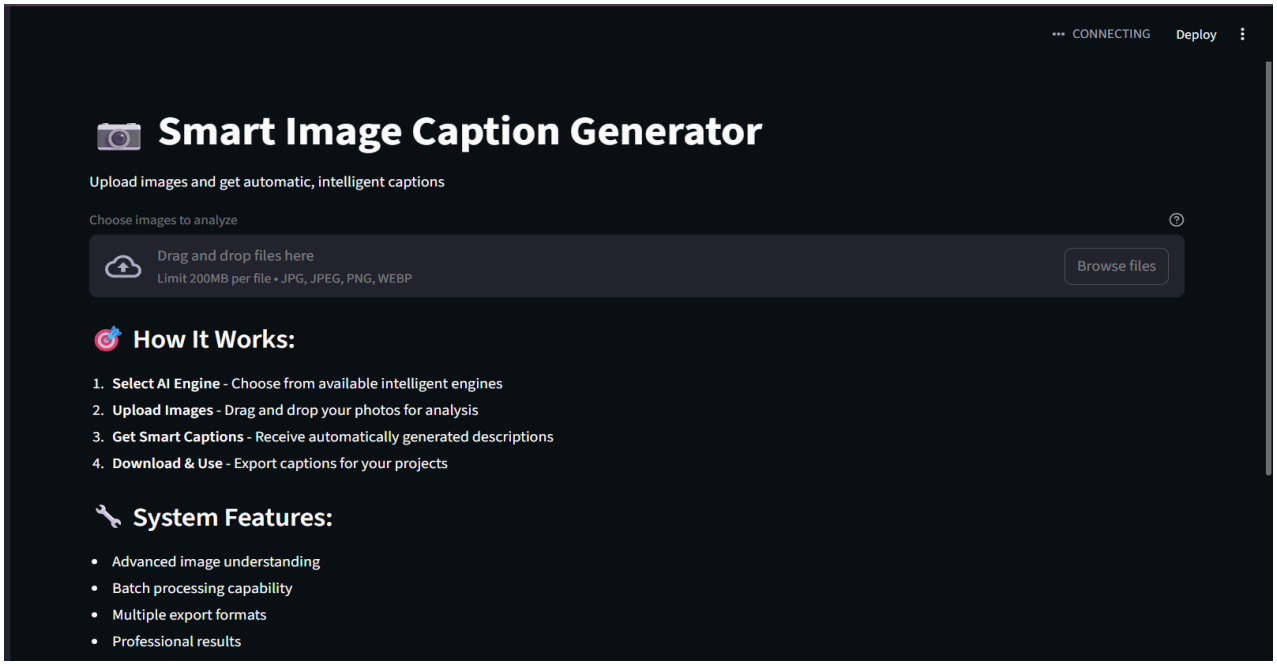
```
import matplotlib.pyplot as plt

plt.figure(figsize=(20, 8))
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Model Loss During Training', fontsize=16)
plt.xlabel('Epoch', fontsize=14)
plt.ylabel('Loss', fontsize=14)
plt.legend(loc='upper right', fontsize=12)
plt.grid(True)
plt.show()
```

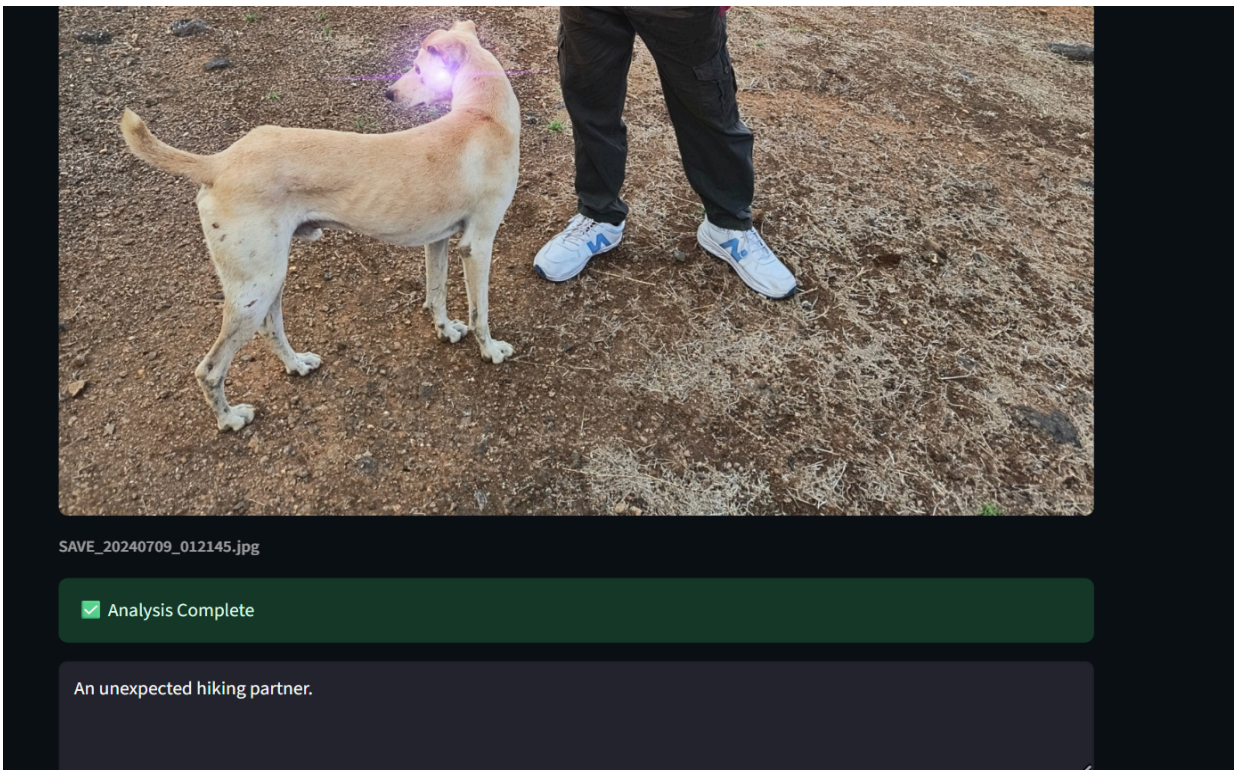


Results & Observations

The model successfully generates semantically relevant captions:

- Example: *"A dog is jumping over a wooden log"* for a dog jumping image.
- Handles common objects well but produces shorter captions.





Observations:

- CNN-LSTM synergy ensures both visual understanding and grammatical fluency.
- Dataset quality improves sentence diversity.
- Dropout and early stopping help reduce overfitting.
- Training is computationally intensive.

Conclusion:

The project demonstrates the integration of Computer Vision and NLP for automatic image captioning. The CNN-LSTM architecture generates human-like captions, providing an end-to-end workflow from preprocessing to deployment. It highlights how AI can bridge the gap between visual understanding and language generation.

Overall, this project demonstrates a practical end-to-end deep learning workflow: from data preprocessing to model training, evaluation, and interactive deployment. It not only showcases the potential of multimodal AI but also provides a foundation for further research in accessibility, automated content annotation, and intelligent human-AI interaction. The integration of NLP and CV in this project exemplifies how AI systems can interpret the world in a manner closer to human understanding, making it a significant step towards intelligent, context-aware applications.

Future Work

Future improvements could include replacing LSTMs with Transformer-based models like ViT and GPT/BERT for better context understanding, and adding attention mechanisms to focus on important regions of an image while generating captions. Training on larger datasets such as MS COCO would improve vocabulary and handle rare objects better. Additionally, deploying the model as a web or mobile application for real-time captioning and extending it to multilingual support would enhance accessibility and practical applications in areas like social media, accessibility tools, and intelligent human-AI interaction.